

FIT1056 – Software Engineering

Software Requirements Specification (SRS)

[PROJECT NAME]

Team / Group:	[GROUP ID / TEAM NAME]
Team Members:	[NAME 1, NAME 2, NAME 3, NAME 4]
Tutor / Workshop:	[DETAILS]
Deliverable:	[Deliverable 1 / Deliverable 2]
Version:	[v1.0]
Date:	August 19, 2026

Document purpose. This Software Requirements Specification (SRS) progressively documents the problem, stakeholders, requirements, design decisions, evaluation evidence, testing strategy, and implementation status of the project across Deliverables 1 and 2. It should be sufficiently clear and traceable that another software engineering team could understand what is being built, why it is being built, and how the design and implementation satisfy the requirements.

Document Control

Table 1: Document revision history

Version	Date	Author(s)	Summary of changes
0.1	[date]	[names]	Initial draft.
0.2	[date]	[names]	Updated after tutor/stakeholder feedback.
1.0	[date]	[names]	Final submission version.

Table 2: Terminology and abbreviations

Term	Meaning in this project
SRS	Software Requirements Specification.
FR	Functional Requirement.
NFR	Non-Functional Requirement.
MVP	Minimum Viable Product.
Project term	Project-specific definition.
.....	
.....	
.....	

Contents

Document Control	1
1 Introduction	4
1.1 Purpose	4
1.2 Intended Audience and Reading Suggestions	4
1.3 Project Scope and Boundaries	4
2 Problem Statement	4
2.1 Current Situation	4
2.2 Problem Impact	4
2.3 Proposed System Goal	4
3 Stakeholder Analysis	5
3.1 Stakeholder List & General Overview	5
3.1.1 Stakeholder Influence–Importance Matrix	5
3.2 Personas	5
3.2.1 Persona 2 – [Role / Persona Name]	6
3.2.2 Persona 3 – [Role / Persona Name]	6
3.2.3 Persona ... – [Role / Persona Name]	6
3.2.4 Persona ... – [Role / Persona Name]	6
3.3 Interview Outcome	6
3.3.1 Interview Setup	6
3.3.2 Key Findings	7
3.3.3 Stakeholder Interaction Scenarios	7

4	System Features and Requirements	7
4.1	System Features	7
4.2	Functional Requirements	7
4.2.1	Feature F-01 – [Feature Name]	7
4.3	Non-functional Requirements	8
4.4	Acceptance Criteria	8
4.5	Implementation Planning	9
5	Use Case Diagram	9
5.1	Use Case Descriptions	10
6	Class Diagram	10
6.1	Class Design Rationale	11
7	System Context Analysis	11
7.1	Operational Context	11
7.2	Behavioural Context	11
7.3	Ecological Context	11
7.4	Sociocultural Context	11
7.5	System Context Diagram	12
7.6	Context-to-Requirement Implications	12
8	System Architecture	12
8.1	Architecture Overview	13
8.2	Components / Layers	13
8.3	Architecture Style Rationale	13
8.4	NFR-driven Design Decisions and Trade-offs	13
9	Usability Evaluation	14
9.1	Evaluation Method	14
9.2	Heuristic Evaluation	14
9.3	Think-Aloud / User Testing Scenarios	14
9.4	Results and Critical Analysis	14
10	Sequence Diagrams	15
10.1	Sequence Diagram 1 – [Use Case / Team Member]	15
10.2	Sequence Diagram 2 – [Use Case / Team Member]	15
11	Quality Assurance Plan	15
11.1	QA Strategy and Test Environment	15
11.2	Unit Testing with PyTest	16
11.2.1	Example Test Code	16
11.3	Functional / End-to-End Testing	16
11.4	Test Coverage and Defect Summary	17
12	Implementation and MVP Evidence	17
12.1	Implemented MVP Features	17
12.2	Key Code Excerpts	17
12.3	Error Handling and Fault Tolerance	17
12.4	Code Quality and Repository Structure	18
12.5	Known Issues / Limitations	18
13	Deliverable 1 Updates and Change Log	18
14	Requirements–Design–Test Traceability	18

15 Conclusion	20
References	20
A Interview Protocol / Question Set	20
B Additional Test Evidence	20
C Additional Code Listings	20
D Supporting Figures / Artefacts	20

List of Figures

1	Stakeholder influence–importance matrix	5
2	Use case diagram for [major actor(s)]	10
3	UML class diagram	11
4	System context diagram showing the system boundary and external entities	12
5	System architecture diagram	13
6	Sequence diagram for [use case]	15

List of Tables

1	Document revision history	1
2	Terminology and abbreviations	1
3	Stakeholder overview	5
4	Persona summary – [Persona 1]	6
5	Interview findings and resulting design implications	7
6	Feature scope and implementation plan	7
7	Functional requirements for F-01	8
8	Non-functional requirements	8
9	Acceptance criteria	8
10	Planned implementation technologies	9
11	Use case specification – UC-01	10
12	Context analysis and design implications	12
13	Architecture components and responsibilities	13
14	Architecture decisions and trade-offs	13
15	Usability heuristic evaluation	14
16	Usability testing scenarios	14
17	Usability findings	14
18	Unit test plan and results	16
19	Functional test cases mapped to requirements	16
20	Test coverage summary	17
21	Implementation status of features	17
22	Known implementation issues and limitations	18
23	Deliverable 1 change log	18
24	Requirements traceability matrix	19

1 Introduction

1.1 Purpose

Student guidance – remove before final submission

State why this document exists and what it specifies. Identify the software product and the scope of this SRS. Keep this short and project-specific. Do not simply repeat the assignment brief.

[Write 1–2 concise paragraphs explaining the purpose of the software and the purpose of this SRS.]

1.2 Intended Audience and Reading Suggestions

Student guidance – remove before final submission

Identify who should be able to use this document: development team, stakeholders, tutors/assessors, future maintainers, testers, or project owners. Briefly tell different readers which sections are most relevant to them.

[Example structure: “Developers should focus on Sections 4–8; testers should also consult Sections 4.4, 11 and 14; stakeholders may begin with Sections 2–4.”]

1.3 Project Scope and Boundaries

Student guidance – remove before final submission

Give a concise statement of what the product will and will not attempt to solve this semester. Keep this high-level; the detailed in-scope/out-of-scope feature decisions belong in Section 4.1.

[Summarise the product boundary, intended users, deployment context, and important constraints.]

2 Problem Statement

Student guidance – remove before final submission

Describe the real-world problem before describing your solution. A strong problem statement identifies: (1) who experiences the problem, (2) what the current difficulty or unmet need is, (3) why it matters, (4) relevant constraints/context, and (5) the opportunity your proposed system addresses. Avoid turning this section into a feature list.

2.1 Current Situation

[Describe the current process or situation and the pain points.]

2.2 Problem Impact

[Explain the consequences for users/stakeholders if the problem remains unresolved.]

2.3 Proposed System Goal

[State the project goal in outcome-oriented language without prematurely prescribing every implementation detail.]

3 Stakeholder Analysis

3.1 Stakeholder List & General Overview

Student guidance – remove before final submission

Cover all major stakeholders and actors. Distinguish between a stakeholder (has an interest/influence) and an actor (interacts directly with the system) where useful. For each stakeholder, state their role, needs, relationship with the system, influence, and importance.

Table 3: Stakeholder overview

Stakeholder	Actor?	Goals / needs	Relationship with system	Influence	Importance
Stakeholder A	Yes/No	[Needs]	[Relationship]	High	High
Stakeholder B	Yes/No	[Needs]	[Relationship]	Med	Low

3.1.1 Stakeholder Influence–Importance Matrix

Student guidance – remove before final submission

Place stakeholders in the four quadrants and justify the positioning. The matrix should inform how you communicate with and prioritise different stakeholder groups.

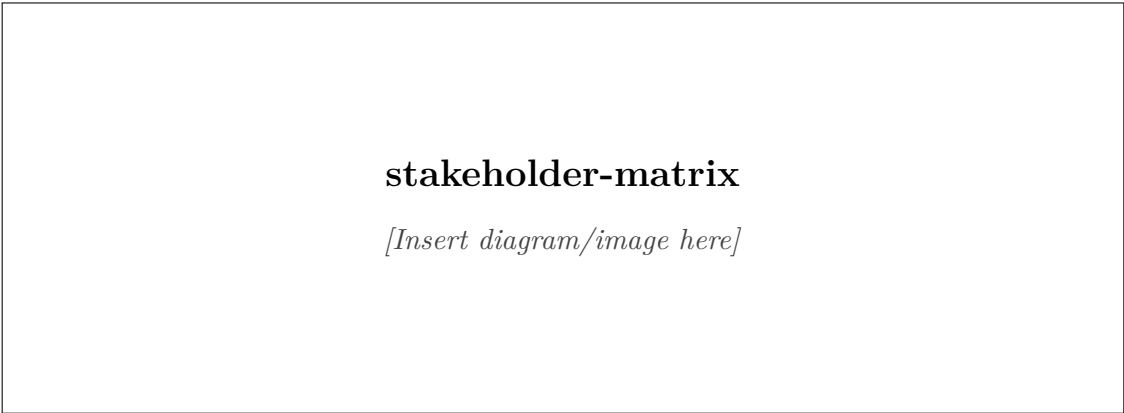


Figure 1: Stakeholder influence–importance matrix

3.2 Personas

Student guidance – remove before final submission

Create one distinct persona per team member, each representing a relevant actor/stakeholder group. A persona should be evidence-informed, not a fictional biography for its own sake. Include goals, frustrations, behaviours, relevant context, accessibility/cultural considerations, and how the person would interact with the proposed system.

Table 4: Persona summary – [Persona 1]

Attribute	Details
 Persona Portrait	
Role / context	[Details]
Goals	[Details]
Needs	[Details]
Pain points / frustrations	[Details]
Digital confidence / behaviours	[Details]
Accessibility, ethical, cultural or privacy considerations	[Details]
Typical interaction scenario	[Details]

3.2.1 Persona 2 – [Role / Persona Name]

[Repeat the persona format. Add one persona per team member.]

3.2.2 Persona 3 – [Role / Persona Name]

[Repeat the persona format. Add one persona per team member.]

3.2.3 Persona ... – [Role / Persona Name]

[Repeat the persona format. Add one persona per team member.]

3.2.4 Persona ... – [Role / Persona Name]

[Repeat the persona format. Add one persona per team member.]

Add additional personas, if needed....

3.3 Interview Outcome**Student guidance – remove before final submission**

Report the outcome of at least one stakeholder-style interview. State who the participant represented, why this persona/stakeholder was selected, the scenario used, key findings, and most importantly what requirements or design decisions changed because of the interview. Do not include unnecessary personal data.

3.3.1 Interview Setup

[Participant role/persona, date, method, scenario/tasks, and ethical/privacy considerations.]

3.3.2 Key Findings

Table 5: Interview findings and resulting design implications

ID	Finding / observation	Requirement or design implication	Linked ID(s)
I1	[Finding]	[Implication]	FR-01 / NFR-02
I2	[Finding]	[Implication]	FR-04
<i>Add additional interview findings below as needed:</i>			
I3	...	...	...
I4	...	...	...
I5	...	...	...

3.3.3 Stakeholder Interaction Scenarios

[Describe the realistic scenario(s) in which the interviewed stakeholder would interact with the system.]

4 System Features and Requirements

4.1 System Features

Student guidance – remove before final submission

List the major user-visible or system-level features. Clearly distinguish features planned for the semester from features intentionally left out of scope. The scope must be realistic for the team's time and resources.

Table 6: Feature scope and implementation plan

ID	Feature	Description / user value	Scope	Priority / milestone
F-01	[Feature]	[Description]	In scope	Must / MVP
F-02	[Feature]	[Description]	In scope	Should
F-03	[Feature]	[Description]	Out of scope	Future
<i>Add additional project features below:</i>				
F-04	...	...	...	...

4.2 Functional Requirements

Student guidance – remove before final submission

Write testable, unambiguous requirements using consistent “shall” language. Organise requirements by feature. Each requirement should express one observable capability or behaviour, include a rationale where useful, and avoid combining multiple requirements into one sentence.

4.2.1 Feature F-01 – [Feature Name]

Table 7: Functional requirements for F-01

ID	Requirement	Rationale / source
FR-01	The system shall [single testable behaviour].	[Why needed / stakeholder source].
FR-02	The system shall [single testable behaviour].	[Why needed / stakeholder source].
<i>Add additional functional requirements below:</i>		
FR-03	...	...

4.3 Non-functional Requirements

Student guidance – remove before final submission

Select NFR classes that genuinely matter to your product (for example usability, privacy/security, performance, reliability, maintainability, accessibility, portability). Explain why each class matters, then write measurable “shall” requirements. Avoid vague statements such as “the system shall be user-friendly.” Quality models such as ISO/IEC 25010 may be cited where relevant [1].

Table 8: Non-functional requirements

ID	Category	Requirement	Rationale / measurement
NFR-01	Usability	The system shall [measurable usability expectation].	[How it will be evaluated].
NFR-02	Privacy	The system shall [specific privacy behaviour].	[Rationale and evidence].
NFR-03	Reliability	The system shall [measurable reliability behaviour].	[How tested].
<i>Add additional non-functional requirements below:</i>			
NFR-04	...	...	...

4.4 Acceptance Criteria

Student guidance – remove before final submission

Define SMART acceptance criteria that demonstrate when each feature is acceptable. Include enough criteria to cover normal, invalid, and edge behaviours. For FIT1056 Deliverable 1, aim for at least five meaningful acceptance criteria per feature. Each criterion should trace to a feature and one or more requirements.

Table 9: Acceptance criteria

AC ID	Feature	Req. ID(s)	Acceptance criterion	Evidence / status
AC-01	F-01	FR-01	Given [precondition], when [action], then [observable result].	[D2: test ID / result]
AC-02	F-01	FR-02	[Specific measurable criterion].	[D2: evidence]

AC ID	Feature	Req. ID(s)	Acceptance criterion	Evidence / status
<i>Add additional acceptance criteria below:</i>				
AC-03	...	...	...	...

4.5 Implementation Planning

Student guidance – remove before final submission

For Deliverable 1, describe the intended implementation approach at a planning level: language/frameworks, data storage, major libraries, interfaces, and important constraints. Do not present unimplemented code as completed work. In Deliverable 2, revise this subsection to reflect the actual implementation.

Table 10: Planned implementation technologies

Area	Technology / approach	Rationale and constraints
Programming language	Python [version]	[Rationale]
User interface	[CLI / Tkinter / Streamlit / etc.]	[Rationale]
Data storage	[CSV / JSON / SQLite / etc.]	[Rationale]
Testing	PyTest	[Planned use]
Version control	Git	[Branching / collaboration approach]
<i>Add additional architectural components or tools below:</i>		
...	...	...

5 Use Case Diagram

Student guidance – remove before final submission

Provide use case diagrams covering all major actors and major system operations. If one diagram becomes unreadable, use multiple diagrams grouped by actor or subsystem. Every diagram must be described in the text and must be consistent with the functional requirements.



Figure 2: Use case diagram for [major actor(s)]

5.1 Use Case Descriptions

Table 11: Use case specification – UC-01

Field	Description
Use case ID / name	UC-01 – [Name]
Primary actor	[Actor]
Goal	[Goal]
Preconditions	[Preconditions]
Trigger	[Trigger]
Main success flow	1. The actor initiates the process. 2. The system validates the request. 3. The system displays the output.
Alternative / excep- tion flows	[Invalid input, unavailable data, cancellation, etc.]
Postconditions	[Observable state after completion]
Linked require- ments	FR-01, FR-02
<i>Add additional custom specification fields below:</i>	
...	...

6 Class Diagram

Student guidance – remove before final submission

Include an initial class model if required for your Deliverable 1 workflow, then refine it for Deliverable 2. The final class diagram should be consistent with the requirements and architecture and show relevant classes, attributes/operations, multiplicities, and appropriate UML relationships such as associations, aggregations, compositions, and inheritance. Avoid adding classes merely to make the diagram look complex.



Figure 3: UML class diagram

6.1 Class Design Rationale

[Explain the major classes, their responsibilities, important relationships, and how this design supports the requirements and architecture.]

7 System Context Analysis

Student guidance – remove before final submission

Analyse the system in all four FIT1056 context perspectives: operational, behavioural, ecological, and sociocultural. Also consider relevant physical, business, legal/regulatory, ethical, organisational, and external-system constraints where applicable. The context analysis should influence requirements and design decisions rather than remain a standalone description.

7.1 Operational Context

[Where, when, by whom, and under what operational conditions will the system be used?]

7.2 Behavioural Context

[What user behaviours, workflows, habits, mistakes, or interaction patterns affect the system?]

7.3 Ecological Context

[What surrounding systems, devices, infrastructure, physical settings, organisational processes, or environmental factors interact with the system?]

7.4 Sociocultural Context

[What social, cultural, accessibility, ethical, language, trust, privacy, or inclusion considerations matter?]

7.5 System Context Diagram

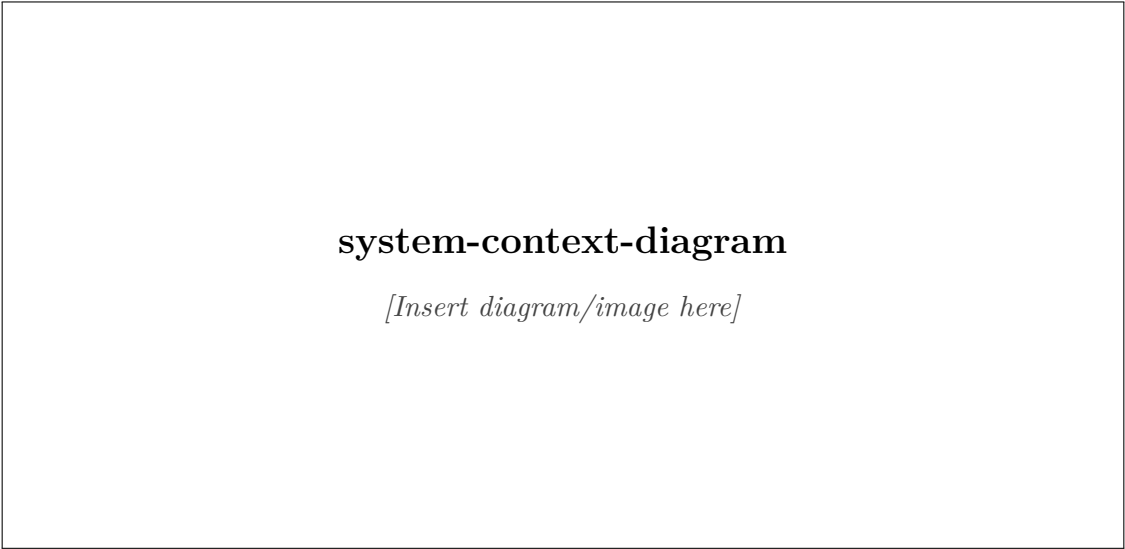


Figure 4: System context diagram showing the system boundary and external entities

7.6 Context-to-Requirement Implications

Table 12: Context analysis and design implications

Context	Observation / constraint	Design or requirement implication	Linked ID(s)
Operational	[Observation]	[Implication]	NFR-01
Sociocultural	[Observation]	[Implication]	FR-04
Add additional context analyses below (e.g., Technical, Physical, Legal, Economic):			
...	...	...	...

8 System Architecture

Student guidance – remove before final submission

Visualise the software architecture using an appropriate architecture style discussed in the unit. Explain each component/layer and justify why the chosen style supports your functional and non-functional requirements. Explicitly discuss important design trade-offs rather than only naming the style.

8.1 Architecture Overview

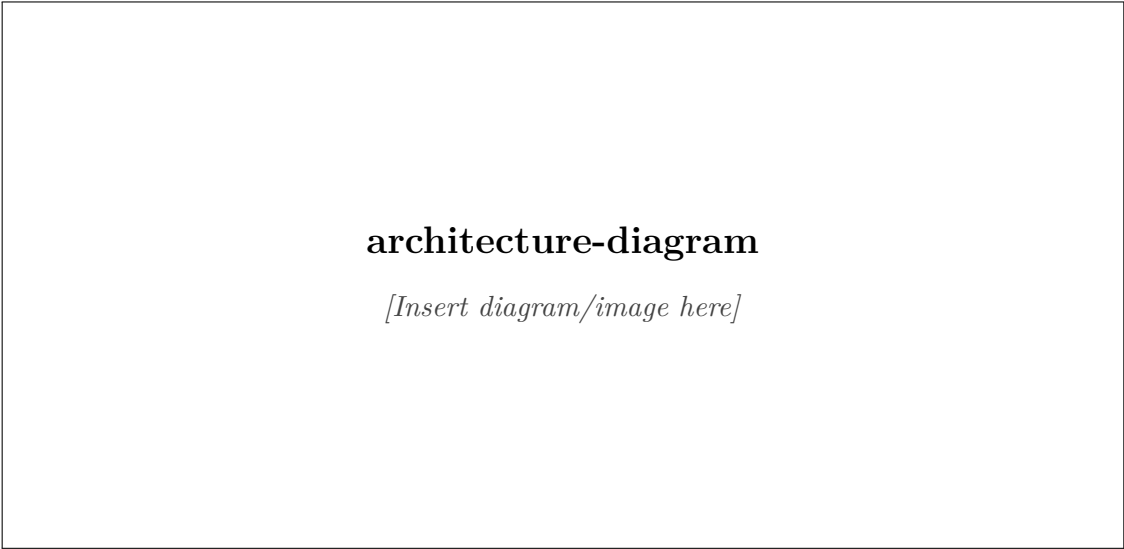


Figure 5: System architecture diagram

8.2 Components / Layers

Table 13: Architecture components and responsibilities

Component / layer	Responsibility	Linked requirements / design rationale
Presentation	[Responsibility]	FR-01, NFR-01
Application / service	[Responsibility]	FR-02, NFR-03
Data	[Responsibility]	FR-03, NFR-02
<i>Add additional architecture subsystems or utility modules below:</i>		
...	...	...

8.3 Architecture Style Rationale

[Explain the selected style and why it is appropriate for this system.]

8.4 NFR-driven Design Decisions and Trade-offs

Table 14: Architecture decisions and trade-offs

NFR	Decision	Benefit	Trade-off / limitation
NFR-01	[Decision]	[Benefit]	[Trade-off]
NFR-02	[Decision]	[Benefit]	[Trade-off]
<i>Log additional architectural decisions and engineering trade-offs below:</i>			
...	...	...	...

9 Usability Evaluation

Student guidance – remove before final submission

Deliverable 2 section. Evaluate your implemented software, not an imagined interface. Cover the usability heuristics taught in the unit and conduct structured user/think-aloud testing from relevant persona perspectives. Report scenarios, observations, strengths, problems, severity where useful, and concrete improvements. Nielsen’s heuristics may be cited as background [2].

9.1 Evaluation Method

[Participants/personas, evaluation method, environment, tasks, data captured, and limitations.]

9.2 Heuristic Evaluation

Table 15: Usability heuristic evaluation

No.	Heuristic	Evidence / observation	Severity	Improvement
1	Visibility of system status	[Observation]	[0–4]	[Improvement]
2	Match between system and real world	[Observation]	[0–4]	[Improvement]
<i>Continue for all heuristics required in the unit.</i>				

9.3 Think-Aloud / User Testing Scenarios

Table 16: Usability testing scenarios

Task ID	Persona / user	Scenario and task	Success measure
UT-01	[Persona]	[Task]	[Completion, errors, time, comments, etc.]
UT-02	[Persona]	[Task]	[Success measure]
<i>Add additional usability testing tasks and metrics below:</i>			
...	...	...	...

9.4 Results and Critical Analysis

Table 17: Usability findings

ID	Observation / user comment	Impact	Action taken / proposed	Linked req.
UF-01	[Finding]	[Low/Med/High]	[Improvement]	NFR-01
<i>Log additional usability testing findings and iteration steps below:</i>				
...	...	...	...	...

10 Sequence Diagrams

Student guidance – remove before final submission

Deliverable 2 section. Each team member should contribute a sequence diagram derived from a meaningful use case. Each diagram should contain the significant interactions needed to explain the behaviour, including 3–4 or more relevant lifelines where appropriate and at least one meaningful combined fragment where required by the unit. Keep messages consistent with the implemented classes and methods.

10.1 Sequence Diagram 1 – [Use Case / Team Member]

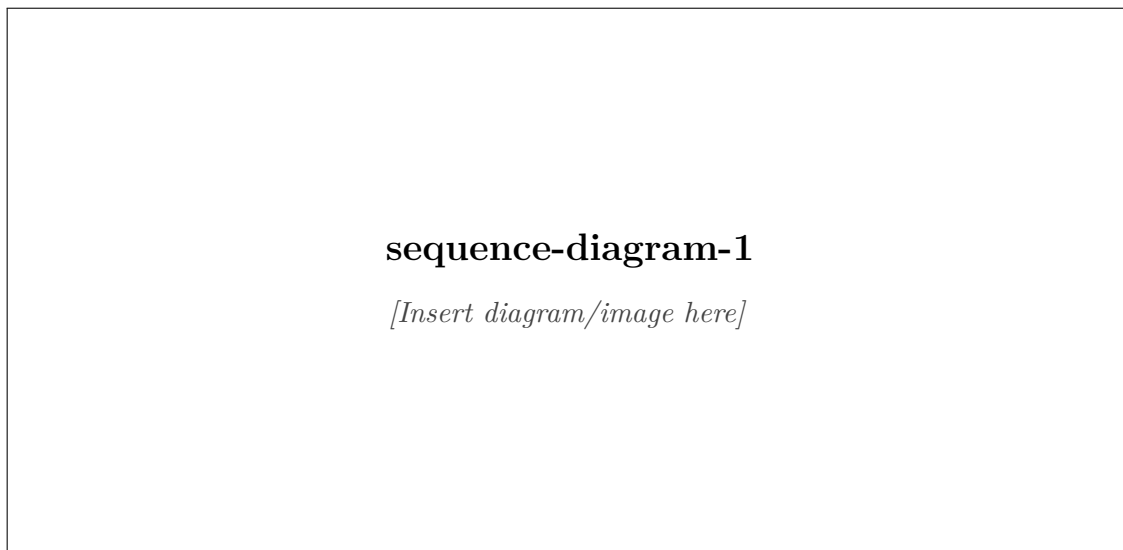


Figure 6: Sequence diagram for [use case]

[Describe what the diagram demonstrates, its main and alternative flow, and which requirements/use cases it realises.]

10.2 Sequence Diagram 2 – [Use Case / Team Member]

[Repeat one per team member.]

11 Quality Assurance Plan

Student guidance – remove before final submission

Deliverable 2 section. Present a comprehensive QA strategy. Include PyTest unit testing with positive, negative, and edge cases; and functional/end-to-end testing mapped to requirements. Where taught/appropriate, apply equivalence partitioning, boundary value analysis, and MC/DC reasoning. Report both the planned tests and the actual outcomes.

11.1 QA Strategy and Test Environment

[Describe test levels, responsibilities, environment, test data, tools, pass/fail rules, and how defects are recorded.]

11.2 Unit Testing with PyTest

Table 18: Unit test plan and results

Test ID	Type	Function / component	Input / condition	Expected	Result
UTEST-01	Positive	[function]	[valid input]	[expected]	Pass/Fail
UTEST-02	Negative	[function]	[invalid input]	[expected error]	Pass/Fail
UTEST-03	Edge	[function]	[boundary input]	[expected]	Pass/Fail
<i>Log additional unit test definitions and runtime status benchmarks below:</i>					
...	...	...	...	...	...

11.2.1 Example Test Code

```

1 def test_example_positive_case():
2     # Arrange
3     value = 2
4
5     # Act
6     result = value * 2
7
8     # Assert
9     assert result == 4

```

Listing 1: Example PyTest structure. Replace with a project-specific test.

11.3 Functional / End-to-End Testing

Table 19: Functional test cases mapped to requirements

Test ID	Req.	Technique	Scenario / steps	Expected outcome	Result
FTEST-01	FR-01	E2E	[User scenario]	[Outcome]	Pass/Fail
FTEST-02	FR-02	BVA	[Boundary case]	[Outcome]	Pass/Fail
FTEST-03	FR-03	Equiv.	[Equivalence class]	[Outcome]	Pass/Fail
<i>Log additional functional test execution matrices and validation checkpoints below:</i>					
...	...	...	...	...	...

11.4 Test Coverage and Defect Summary

Table 20: Test coverage summary

Area	Planned	Passed	Failed	Comment / remaining risk
Unit tests	[n]	[n]	[n]	[Comment]
Functional tests	[n]	[n]	[n]	[Comment]
Acceptance criteria	[n]	[n]	[n]	[Comment]
<i>Add additional verification scopes or test suites below:</i>				
...	...	...	...	...

12 Implementation and MVP Evidence

Student guidance – remove before final submission

Deliverable 2 section. Explain what was actually implemented and demonstrate alignment with the requirements and architecture. The submitted code itself is assessed separately, so this section should provide concise evidence rather than paste the entire project. Include important code excerpts only when they demonstrate a design decision, error handling, or key logic. Document known issues honestly. The codebase should also include a README, and a requirements.txt file where non-standard packages are used.

12.1 Implemented MVP Features

Table 21: Implementation status of features

Feature	Name	Status	Implementation evidence	Linked tests
F-01	[Feature]	Complete	[Module/screen/function]	FTEST-01
F-02	[Feature]	Partial	[What is complete/missing]	FTEST-03
<i>Log additional software feature implementation states and evidence pathways below:</i>				
...	...	...	...	...

12.2 Key Code Excerpts

```

1 def validate_input(value):
2     """Return True only when the supplied value is acceptable."""
3     if value is None:
4         return False
5     return bool(str(value).strip())

```

Listing 2: Example of a short implementation excerpt. Replace with project code that is worth discussing.

[Explain why this excerpt matters: e.g., error handling, separation of concerns, alignment to architecture, privacy logic, or maintainability.]

12.3 Error Handling and Fault Tolerance

[Describe how invalid inputs, missing data, file/database errors, or other anticipated failures are handled gracefully. Link to NFRs/tests.]

12.4 Code Quality and Repository Structure

```
project-root/
|-- src/           # Application source code
|-- tests/         # PyTest tests
|-- assets/        # Images/data required by the app
|-- README.md      # How to install/run/use the system
|-- requirements.txt # If non-standard packages are required
'-- ...
```

12.5 Known Issues / Limitations

Table 22: Known implementation issues and limitations

ID	Issue / limitation	Severity	Workaround / future action
KI-01	[Issue]	Low/Med/High	[Action]
<i>Log additional engineering limitations, known bugs, and technical debt below:</i>			
...	...	...	...

13 Deliverable 1 Updates and Change Log

Student guidance – remove before final submission

Deliverable 2 section. Do not silently overwrite Deliverable 1. Summarise the important changes made in response to tutor feedback, stakeholder learning, implementation reality, and testing. Explicitly identify requirements/design elements that were changed, removed, deferred, or found not applicable, and explain why.

Table 23: Deliverable 1 change log

ID	Affected item	Previous version	Updated version / decision	Reason / evidence
CH-01	FR-03	[Old]	[New]	[Tutor feedback / test / stakeholder]
CH-02	F-05	In scope	Deferred	[Reason]
<i>Log subsequent deliverable modifications, feedback amendments, and version updates below:</i>				
...	...	...	...	...

14 Requirements–Design–Test Traceability

Student guidance – remove before final submission

This section is strongly recommended for document quality and traceability. Map each important requirement through the design and validation evidence. This makes it easy to verify that requirements are neither forgotten during implementation nor tested without a stated requirement.

Table 24: Requirements traceability matrix

Req. ID	Requirement mary	sum-	Feature	Use case	Design element	Test / accep- tance evidence	Status
FR-01	[Summary]		F-01	UC-01	Class A / Com- ponent A	AC-01, FTEST- 01	Satisfied
NFR-01	[Summary]		F-01	–	UI / architec- ture decision	UT-01, heuristic finding	Satisfied
<i>Map additional system requirements to features, architectural elements, and test parameters below:</i>							
...	...		...	...	...	...	...

15 Conclusion

Student guidance – remove before final submission

Keep this brief. Summarise whether the final system meets its intended purpose, the extent to which the key requirements were satisfied, and the most important remaining limitations or future work. Do not introduce new requirements here.

[1–3 concise paragraphs.]

References

- [1] *Systems and software engineering – systems and software quality requirements and evaluation (square) – product quality model*, International Organization for Standardization, 2011.
- [2] J. Nielsen, *10 usability heuristics for user interface design*, Nielsen Norman Group, Updated explanatory material available from Nielsen Norman Group, 1994.

A Interview Protocol / Question Set

[Include the interview questions or protocol if useful. Avoid unnecessary personal information about participants.]

B Additional Test Evidence

[Optional screenshots, selected test output, or supplementary test tables. Keep the main report readable.]

C Additional Code Listings

Student guidance – remove before final submission

Only include additional code here when it genuinely supports the report. The full software project is submitted separately as the project ZIP/repository; the SRS should not become a printout of the source tree.

D Supporting Figures / Artefacts

[Optional additional UI mock-ups, matrices, diagrams, or screenshots that are referenced from the report.]